

Naval Vessel Monitoring & Communication System (NVMCS)

Project Report — Programming in Java (Flipped Course)
VITyarthi — Build Your Own Project

Student Name	Aatreyee Bhattacharjee
Registration No.	24BEC10003
Program	B.Tech Electronics & Communication Engineering
Institution	Vellore Institute of Technology (VIT), Bhopal
Course	Programming in Java
Project Title	Naval Vessel Monitoring & Communication System
Submission Type	GitHub Repository + Project Report (PDF)

1–2. Introduction

The **Naval Vessel Monitoring & Communication System (NVMCS)** is a self-chosen, original project built for the *Programming in Java* flipped-course evaluation. Rather than using a generic CRUD topic, the project is framed around a domain the author is already engaged with through an ongoing internship focus on naval navigation, sensor, weapon and communication systems — giving the coursework a concrete, real-world flavour while staying fully within the scope of a Java console application.

NVMCS is a **layered, menu-driven command-line application** that lets an operator register vessels, attach sensors to them and record readings, track the readiness of weapon systems, and exchange encrypted messages between vessels. It applies core Java concepts taught in the course — classes and objects, interfaces via layering, collections (HashMap, ArrayList), enums, custom checked exceptions, file I/O, and the `java.time` API — inside a clean model / repository / service / presentation architecture, rather than as one large monolithic `main` method.

Objectives

- Design and implement a multi-module Java application demonstrating object-oriented design.
- Apply a layered architecture (model, repository, service, presentation) for maintainability.
- Persist data reliably across runs without requiring an external database server.
- Demonstrate secure handling of sensitive data (encrypted message bodies, rank-gated actions).
- Provide full design documentation, UML diagrams, and a tested, runnable GitHub repository.

3. Problem Statement

Naval vessels depend on three things working correctly and consistently: onboard sensors that report the physical state of the ship and its environment, weapon systems that must be tracked for operational readiness, and reliable communication between vessels so that orders and situation reports reach the right ship without being exposed in plain text. In many training or simulation contexts these three concerns end up handled by disconnected spreadsheets or ad-hoc scripts, which makes it difficult to get one consistent, auditable picture of a vessel's state, and easy to leak sensitive communication content.

NVMCS addresses this by bringing vessel registration, sensor monitoring, weapon-system readiness tracking, and encrypted inter-vessel messaging together under a single, layered, auditable console application, with every significant action logged to a timestamped audit trail.

Scope note: this is a single-user, offline educational/simulation tool built for a coursework submission. It is not, and is not intended to be, a real military command-and-control system.

Target Users

- Coursework evaluators assessing this submission against the VITyarthi rubric.
- A naval training-exercise operator wanting a lightweight, offline console tool to simulate fleet monitoring and communication.

4. Functional Requirements

The system is organised into four major functional modules, each with a clear input/output structure and a logical, menu-driven workflow:

4.1 Vessel Registry

- **Input:** vessel id, name, class, commanding officer rank.
- **Output:** confirmation of registration; a listable directory of all vessels.

4.2 Sensor Data Management

- **Input:** sensor id, owning vessel, sensor type (SONAR/RADAR/GPS/TEMPERATURE/ PRESSURE/FUEL), warning threshold; later, timestamped readings.
- **Output:** confirmation of registration, a recorded reading, a reading history per sensor, and an automatic WARN-level alert when a reading meets or exceeds the sensor's configured threshold.

4.3 Weapon System Readiness

- **Input:** weapon id, owning vessel, name, ammunition count; later, a status change request (READY / MAINTENANCE / DISABLED / RELOADING).
- **Output:** confirmation of registration, updated status (subject to an access check — see NFRs), a listing of weapons, and a fleet-wide readiness percentage.

4.4 Communication

- **Input:** sender vessel, recipient vessel, message body, priority level (ROUTINE / PRIORITY / FLASH).
- **Output:** confirmation that the message was sent and stored (encrypted); an inbox view per vessel that decrypts messages only at the point of reading.

Overall Workflow

The operator is presented with a numbered main menu; each module has its own numbered sub-menu. Input is validated at the service layer before any state changes are made, and every operation that changes state is persisted immediately and logged. See the workflow diagram in Section 7.

5. Non-Functional Requirements

NFR	How NVMCS addresses it
Performance	Repositories use HashMap/LinkedHashMap keyed by id, giving O(1) average-case lookup instead of linear scans.
Security	Message bodies are passed through SimpleCipher (an XOR-based cipher) before being written to disk, so data/messages.csv never contains plain text; weapon-status changes are gated behind a minimum COMMANDER officer rank check.
Usability	A numbered, self-explanatory CLI menu re-prompts on invalid input instead of crashing or silently failing.
Reliability	Custom checked exceptions (VesselNotFoundException, InvalidSensorDataException, CommunicationException, AccessDeniedException) are caught at the CLI boundary, so bad input or missing records never crash the application.
Scalability	New vessels, sensors, weapons or sensor types can be added purely as data, with no code changes; each layer (model/repository/service/CLI) can be extended or replaced independently.
Maintainability	Strict package-per-concern layering, Javadoc comments on every public class, and single-responsibility service classes keep each unit small and easy to change.
Logging / monitoring	FileLogger appends a timestamped line to data/system.log for every registration, reading, alert, status change and message.
Resource efficiency	All file streams use try-with-resources so handles are always released; no unbounded in-memory growth beyond the current session's records.

6. System Architecture

NVMCS follows a four-layer architecture. The **presentation layer** (NavalMonitoringApp) is the only class that talks to the console; it delegates all business logic to the **service layer**, which in turn delegates persistence to the **repository layer**. All four layers operate on plain **model** objects, and cross-cutting utilities (logging, encryption, validation) are used from the service layer.

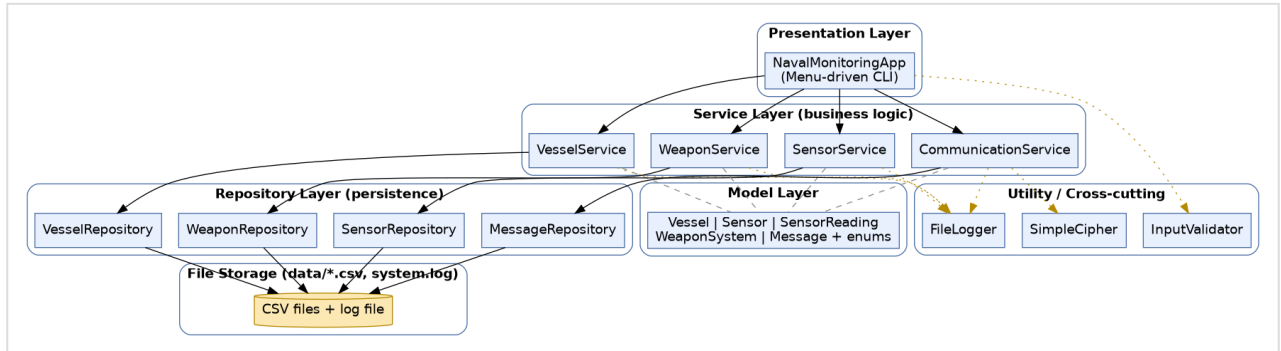


Figure 6.1 — System architecture (layered design)

Why this architecture?

Separating concerns this way means the CLI could be replaced with a GUI or a REST API without touching any business logic, the CSV-based repositories could be swapped for a real database without touching the service layer, and every layer can be unit-tested independently (see Section 11).

7. Design Diagrams

7.1 Process Flow / Workflow Diagram

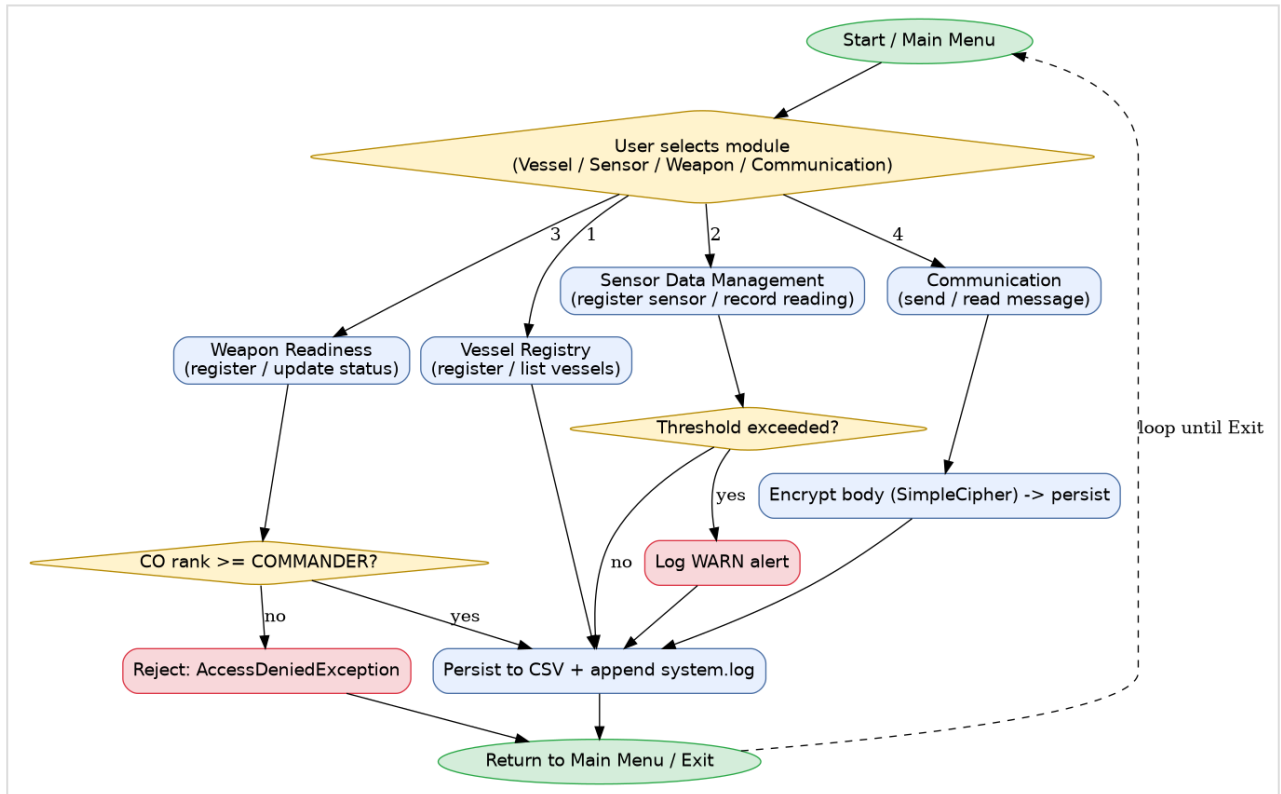


Figure 7.1 — End-to-end operator workflow across all four modules

7.2 Use Case Diagram

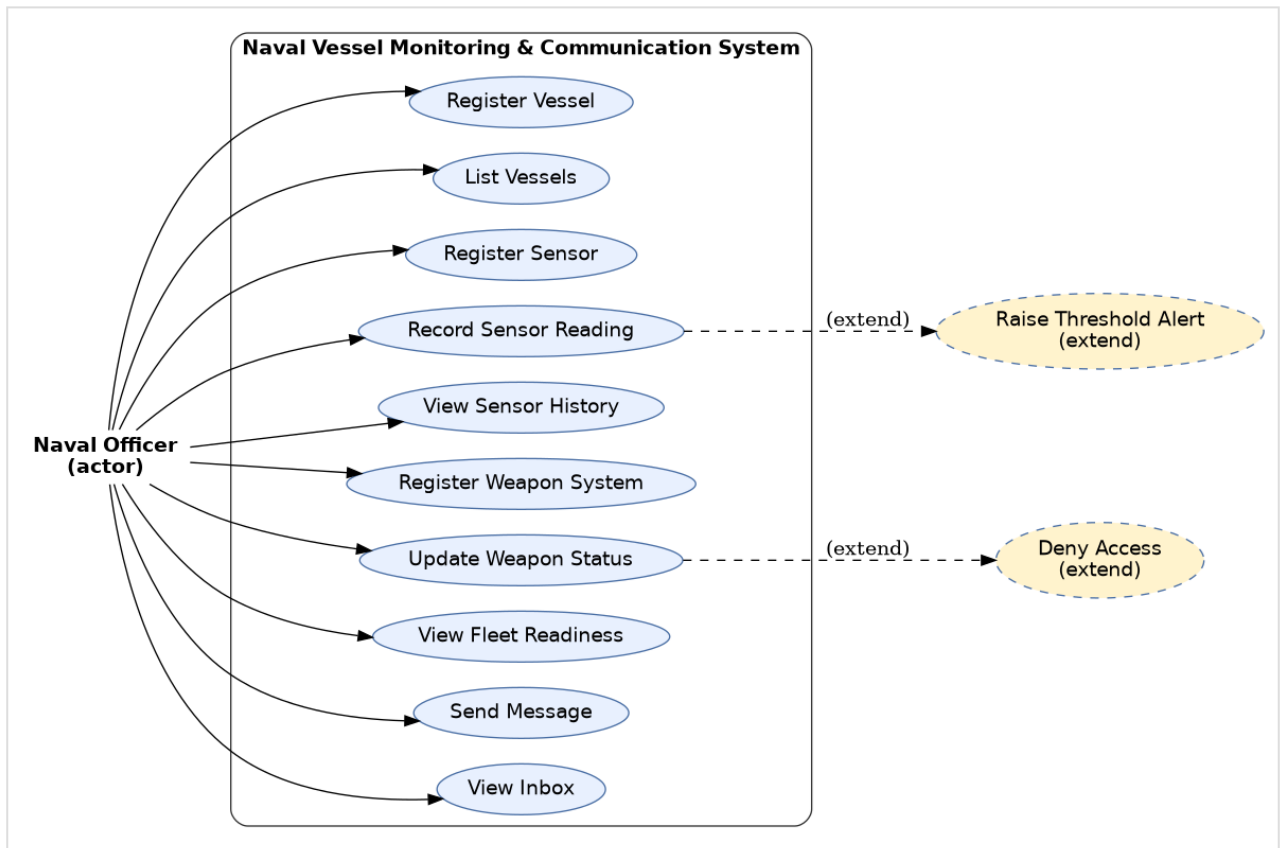


Figure 7.2 — Use cases available to the Naval Officer actor

7.3 Class Diagram

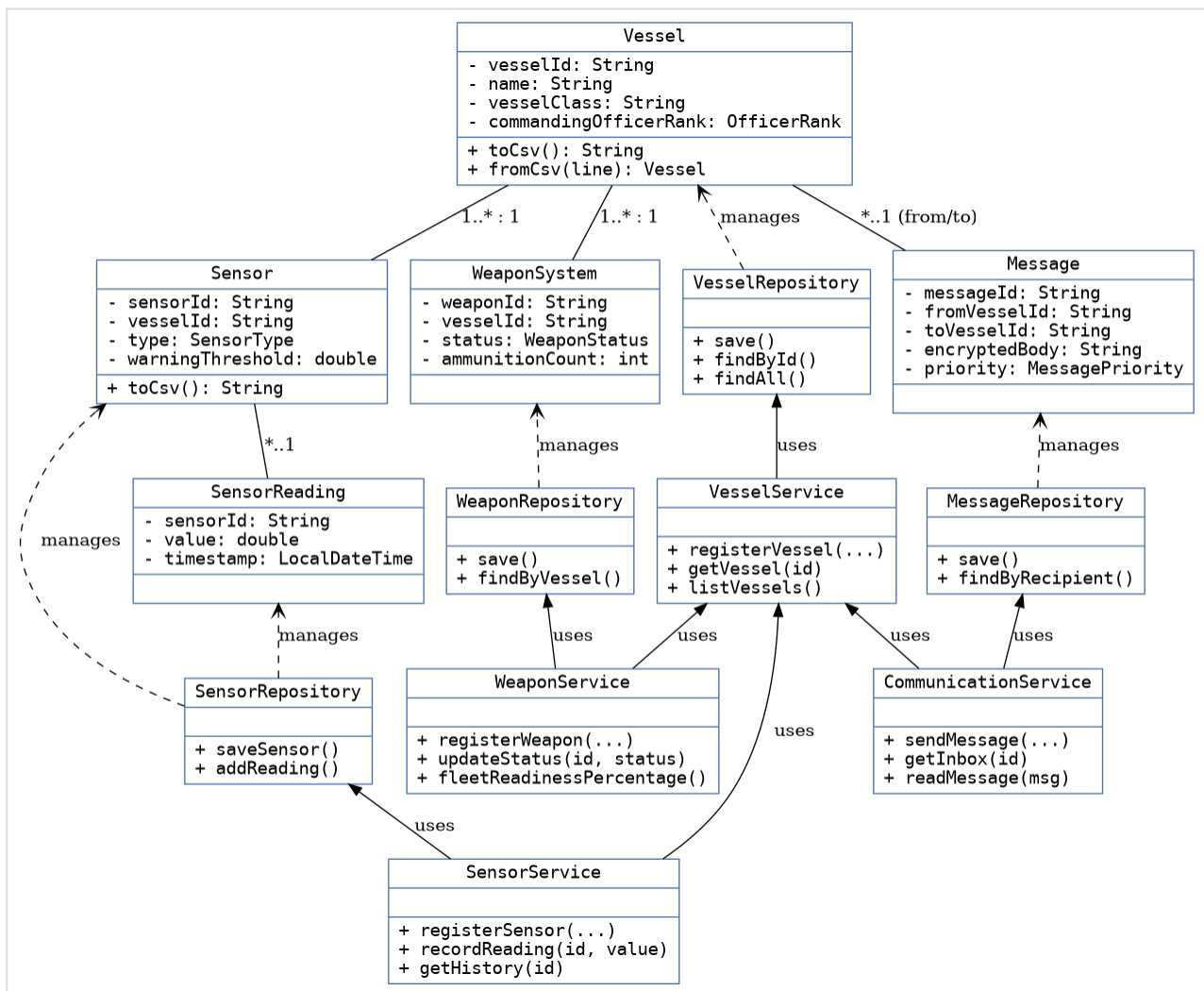


Figure 7.3 — Key model, service and repository classes and their relationships

7.4 Sequence Diagram

The sequence below traces a "record sensor reading" call end to end, including the conditional threshold alert.

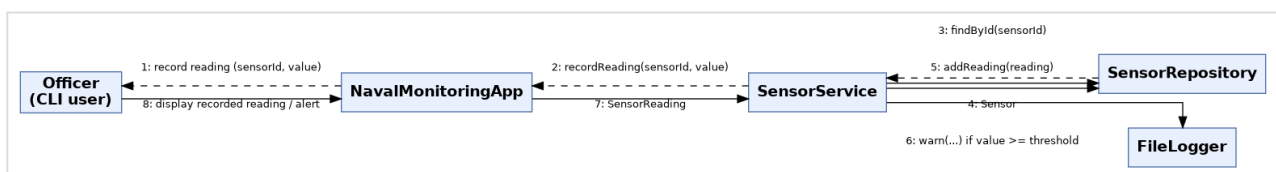


Figure 7.4 — Sequence diagram: recording a sensor reading

7.5 Storage Design

No relational database is used; instead each aggregate is persisted as an append-friendly CSV file under `data/`, which serves the same role a schema/ER diagram would for a database-backed system:

File	Columns
------	---------

vessels.csv	vesselId, name, vesselClass, commandingOfficerRank
sensors.csv	sensorId, vesselId (FK), type, warningThreshold
readings.csv	sensorId (FK), value, timestamp
weapons.csv	weaponId, vesselId (FK), name, status, ammunitionCount
messages.csv	messageId, fromVesselId (FK), toVesselId (FK), encryptedBody, priority, sentAt
system.log	free-text timestamped audit log (not tabular)

8. Design Decisions & Rationale

- **CSV over a real database:** the project brief allows any appropriate storage approach and explicitly makes ER/schema design conditional ("if applicable"). CSV keeps the project runnable from the command line with zero external setup (a hard requirement of the submission), while `toCsv()`/`fromCsv()` methods on each model class keep the serialization logic isolated and easy to swap for JDBC later.
- **Checked exceptions for domain errors:** `VesselNotFoundException`, `InvalidSensorDataException`, `CommunicationException` and `AccessDeniedException` are checked exceptions so the compiler forces every call site to acknowledge these are expected, recoverable outcomes — not bugs.
- **XOR cipher instead of a "real" cryptographic library:** the goal is to demonstrate the *architectural habit* of never persisting sensitive data in plain text, without pulling in an external dependency the "run from the command line, zero setup" requirement would otherwise complicate. This is explicitly documented as an educational simplification, not production-grade cryptography.
- **Rank-gated weapon status changes:** modelled as a simple ordinal comparison on the `OfficerRank` enum, to give a concrete, testable example of an authorization rule rather than leaving "security" as an abstract checkbox.
- **Manual assertion-based tests over JUnit:** the execution environment has no configured build tool or dependency manager, so a small dependency-free test harness (`SensorServiceTest`, run with `java -ea`) was used instead, keeping "clone and run" truly zero-setup while still exercising the core business rules.

9. Implementation Details

The codebase has 21 Java source files across 5 packages, comfortably above the minimum 5–10 meaningful modules/classes requested:

```
com.naval.monitoring
├─ model/          Vessel, Sensor, SensorReading, WeaponSystem, Message,
│                  SensorType, WeaponStatus, MessagePriority, OfficerRank
├─ repository/     VesselRepository, SensorRepository, WeaponRepository, MessageRepository
├─ service/        VesselService, SensorService, WeaponService, CommunicationService
├─ exception/      VesselNotFoundException, InvalidSensorDataException,
│                  CommunicationException, AccessDeniedException
├─ util/           FileLogger, SimpleCipher, InputValidator
└─ NavalMonitoringApp.java (CLI entry point)
```

Key implementation points

- Each service depends only on the repository interfaces it needs and on `VesselService` for cross-references — there is no circular coupling between services.
- `Optional<T>` is used throughout the repository layer for lookups, forcing callers to explicitly handle the "not found" case.
- Java 8+ `java.time.LocalDateTime` is used for all timestamps, formatted with an explicit `DateTimeFormatter` to keep CSV parsing unambiguous.
- Generics are used in the CLI's `promptEnum(String, T[])` helper so a single method drives every enum-based prompt (rank, sensor type, weapon status, priority) without duplicated input-loop code.

Build & Run

```
mkdir -p bin
find src -name "*.java" > sources.txt
javac -d bin @sources.txt
java -cp bin com.naval.monitoring.NavalMonitoringApp
```

10. Screenshots / Results

Below is an annotated transcript of a representative session exercising all four modules, including a threshold alert, a status update, and a full send/receive message round-trip.

```
===== Naval Vessel Monitoring & Communication System =====
1. Vessel Registry    2. Sensor Data Management
3. Weapon System Readiness  4. Communication    0. Exit
Choose an option: 1

-- Vessel Registry --
Vessel id: INS01 | Name: INS Vikrant | Class: Aircraft Carrier | Rank: 4 (CAPTAIN)
Registered: Vessel[id=INS01, name=INS Vikrant, class=Aircraft Carrier, CO-rank=CAPTAIN]

Choose an option: 2 (Sensor Data Management -> Register sensor)
Sensor id: SON01 | Vessel id: INS01 | Type: 1 (SONAR) | Threshold: 75
Registered: Sensor[id=SON01, vessel=INS01, type=SONAR, threshold=75.00]

Choose an option: 2 -> Record reading
Sensor id: SON01 | Reading value: 95
[WARN] ALERT: SONAR reading 95.00 exceeds threshold 75.00 on vessel INS01
Recorded: Reading[sensor=SON01, value=95.00, at=2026-09-17T23:46:29]

Choose an option: 3 (Weapon System Readiness -> Update status)
Weapon id: TORP01 | New status: 2 (MAINTENANCE)
[INFO] Weapon status updated: Weapon[id=TORP01, ..., status=MAINTENANCE, ammo=30]

Choose an option: 4 (Communication -> Send message)
From: INS01 | To: INS02 | Body: "All units maintain formation, proceed heading 090" | Priority: 2
[INFO] Message sent INS01 -> INS02 [PRIORITY]
Message sent (stored encrypted).

Choose an option: 4 -> View inbox
Vessel id: INS02
[2026-09-17T23:46:29] from INS01 (PRIORITY): All units maintain formation, proceed heading 090

Choose an option: 0
Goodbye.
```

Proof of encryption at rest

The corresponding raw line written to `data/messages.csv` — the plaintext body is never written to disk, only the hex-encoded ciphertext:

```
b154b3d5,INS01,INS02,1B36367A2F34332E297A373B33342E3B33347A3C3528373B2E333534767A2A2835393F3F3E7A323F3B3E33343D7A6A636A,PRIORITY,2026-09-17T23:46:29.967335355
```

11. Testing Approach

Because the execution environment has no dependency manager configured, testing is done with a small, dependency-free assertion-based harness, `SensorServiceTest`, run with Java's built-in assert mechanism (`-ea` flag):

Test	What it verifies
<code>testVesselRegistrationAndLookup</code>	A registered vessel can be retrieved by id and its fields round-trip correctly.
<code>testSensorThresholdAlertLogic</code>	A normal reading is recorded correctly; recording against an unknown sensor id throws <code>InvalidSensorDataException</code> .
<code>testWeaponAccessControl</code>	A vessel commanded by a CADET cannot change a weapon's status — <code>AccessDeniedException</code> is thrown as expected.
<code>testMessageEncryptionRoundTrip</code>	Encrypting then decrypting a message body returns the original text, and the encrypted form differs from the plaintext.

```
$ javac -d bin @sources.txt
$ javac -d bin -cp bin test/com/naval/monitoring/SensorServiceTest.java
$ java -ea -cp bin com.naval.monitoring.SensorServiceTest
[PASS] testVesselRegistrationAndLookup
[PASS] testSensorThresholdAlertLogic
[PASS] testWeaponAccessControl
[PASS] testMessageEncryptionRoundTrip
ALL TESTS PASSED
```

In addition, the full CLI was exercised manually end-to-end (Section 10) across all four modules, including the negative/error paths: an unknown sensor id, an out-of-range menu choice, and an access-denied weapon status change.

12. Challenges Faced

- **Testing without a build tool:** the project brief expects "testing wherever applicable" but the environment has no Maven/Gradle/JUnit configured. This was solved by writing a minimal assertion-based harness that needs nothing beyond the JDK itself (`java -ea`), keeping the "clone and run" instructions genuinely dependency-free.
- **Keeping CSV persistence simple but safe:** naive CSV writing breaks if a field contains a comma. The design decision was to disallow commas at the input-validation layer (`InputValidator`) rather than adding a full CSV-escaping library, which was judged out of scope for the project's size.
- **Demonstrating "security" honestly:** it would have been easy to claim "encryption" without it meaning anything. Instead the report is explicit that `SimpleCipher` is an XOR-based, educational demonstration of the *architectural pattern* of never persisting sensitive data in plain text — not a claim of production-grade cryptographic strength.

13. Learnings & Key Takeaways

- Layering a Java application by responsibility (model / repository / service / CLI) makes it much easier to reason about where a bug or a new feature belongs, compared to putting logic directly inside `main`.
- Checked exceptions, used deliberately for a small number of well-named domain errors, make the code's failure modes self-documenting at the call site.
- "Non-functional requirements" stop being abstract once you have to point at one line of code that satisfies each one — e.g. the exact `HashMap` that gives $O(1)$ lookup, or the exact `SimpleCipher.encrypt()` call that keeps a message off disk in plain text.

14. Future Enhancements

- Replace CSV persistence with a real embedded database (e.g. SQLite via JDBC) while keeping the repository interfaces unchanged.
- Replace `SimpleCipher` with an actual AES implementation via `javax.crypto` for genuine confidentiality.
- Add a REST API layer (e.g. with a lightweight framework) on top of the existing service layer, reusing it without modification, to allow a web or mobile front-end.
- Add multi-user login so that the current "single CLI session" model becomes a properly authenticated, per-officer session with an audit trail of who performed each action.

15. References

1. Oracle, *Java SE 21 Documentation* — `java.util`, `java.time`, `java.io` packages.
2. VITyarthi, *Build Your Own Project* — *General Project Instructions & Submission Guidelines* (course-provided brief used to scope this project).
3. Course material for *Programming in Java* (CSE2006), VIT Bhopal — object-oriented design, collections, exception handling.